



# Database Systems

## Transaction

Antu Chowdhury  
Lecturer  
Dept of CSE

# Transaction

- A **transaction** is a *unit* of program execution that accesses and possibly updates various data items.
- E.g., transaction to transfer \$50 from account A to account B:
  1. **read(A)**
  2.  **$A := A - 50$**
  3. **write(A)**
  4. **read(B)**
  5.  **$B := B + 50$**
  6. **write(B)**
  7. ***Commit***
- Two main operations: Read and Write

# Transaction

- In **Database Management Systems (DBMS)**, a **transaction** is a sequence of one or more SQL operations (such as read, write, update, or delete) performed as a single logical unit of work. Transactions must follow the **ACID** properties to ensure data integrity and consistency.
- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions
- A **transaction** can consist of:
- **Read operations:** Retrieving data from the database.
- **Write operations:** Modifying or inserting data into the database.

**COMMIT:** Saves the changes made by the transaction permanently.

**ROLLBACK:** Undoes all changes made by the transaction if an error occurs.

# ACID Properties of Transaction



ATOMICITY



CONSISTENCY



ISOLATION



DURABILITY

# ACID Properties of Transaction

- **Atomicity.** Either all operations of the transaction are properly reflected in the database or none are.
- **Consistency.** Before a transaction starts, the database is in a **consistent state** (all rules are followed). After the transaction completes, the database **must still be consistent** — if not, the transaction is rolled back.

**Isolation.** **Isolation** ensures that when multiple transactions execute concurrently, each transaction  $T_i$  operates independently and does not see the intermediate results of any other transaction  $T_j$ . For any two transactions,  $T_i$  and  $T_j$ , transaction  $T_i$  sees things as if either  $T_j$  finished completely before  $T_i$  started, or  $T_j$  started only after  $T_i$  finished. This means transactions don't interfere with each other, so the database stays accurate and consistent.

- **Durability.** After a transaction completes successfully, the changes it has made to the database persist, even if there are system failures.

# ACID Properties of Transaction

- Transaction to transfer \$50 from account A to account B:
  1. **read**(A)
  2.  $A := A - 50$
  3. **write**(A)
  4. **read**(B)
  5.  $B := B + 50$
  6. **write**(B)
- **Atomicity requirement**
  - If the transaction fails after step 3 and before step 6, money will be “lost” leading to an inconsistent database state
    - Failure could be due to software or hardware
  - The system should ensure that updates of a partially executed transaction are not reflected in the database

# ACID Properties of Transaction

- Transaction to transfer \$50 from account A to account B:
  1. **read**(A)
  2.  $A := A - 50$
  3. **write**(A)
  4. **read**(B)
  5.  $B := B + 50$
  6. **write**(B)
- **Consistency requirement**
  - The total amount must be maintained before or after the transaction.  
The sum of A and B is unchanged by the execution of the transaction.
  - **Total money before transaction = Total money after transaction**
  - Formally:  $A_{\text{(before)}} + B_{\text{(before)}} = A_{\text{(after)}} + B_{\text{(after)}}$

# ACID Properties of Transaction - Isolation

| T1                                      | T2                                                            | Description/Status                         |
|-----------------------------------------|---------------------------------------------------------------|--------------------------------------------|
| Read (A)<br>$A := A - 750$<br>Write (A) |                                                               | Rs 5000<br>Rs 4250<br>Store to A           |
|                                         | Read (A)<br>$temp := A * 0.2$<br>$A := A - temp$<br>Write (A) | Rs 4250<br>Rs 850<br>Rs 3400<br>Store in A |
| Read (B)<br>$B := B + 750$<br>Write (B) |                                                               | Rs 8000<br>Rs 8750<br>Store to B           |
|                                         | Read (B)<br>$B := B + temp$<br>Write (B)                      | Rs 8750<br>Rs 9600<br>Store in B           |



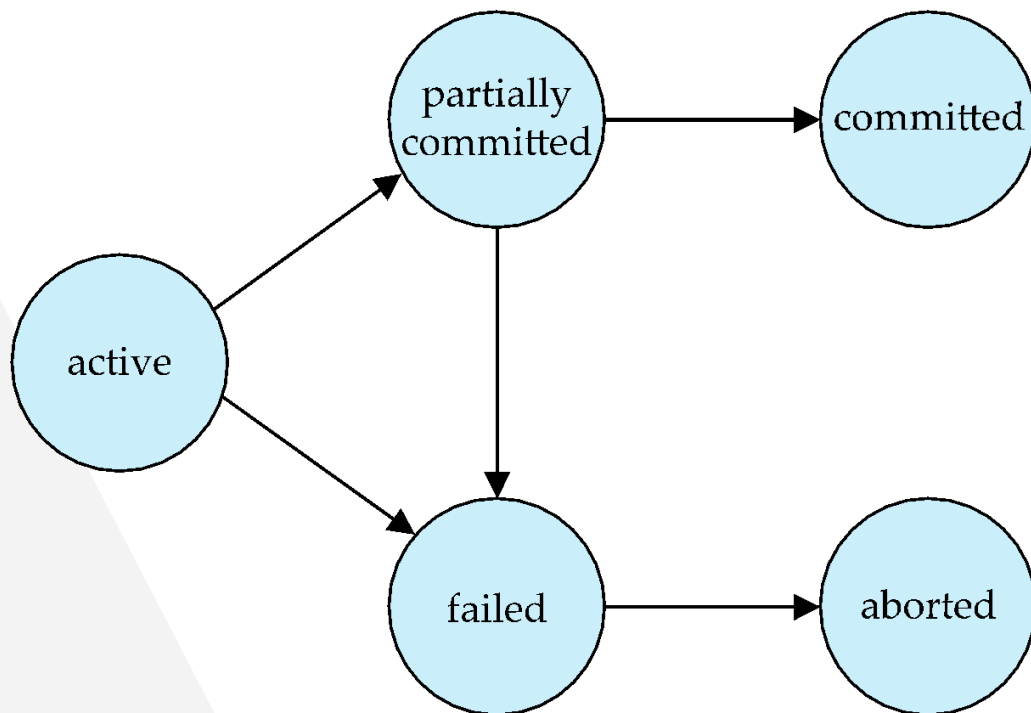
# ACID Properties of Transaction - Isolation

| T1                                                   | T2                                                                        | Description/Status                                            |
|------------------------------------------------------|---------------------------------------------------------------------------|---------------------------------------------------------------|
| Read (A)<br>$A := A - 750$                           |                                                                           | Rs 5000<br>Rs 4250                                            |
|                                                      | Read (A)<br>$temp := A * 0.2$<br>$A := A - temp$<br>Write (A)<br>Read (B) | Rs 5000<br>Rs 1000<br>Rs 4000<br>Store to A = 4000<br>Rs 8000 |
| Write (A)<br>Read (B)<br>$B := B + 750$<br>Write (B) |                                                                           | Store to A = 4250<br>Rs 8000<br>Rs 8750<br>Store to B = 8750  |
|                                                      | $B := B + temp$<br>Write (B)                                              | RS 9000<br>Store to B = 9000                                  |

# Transaction State

- **Active** – the initial state; the transaction stays in this state while it is executing
- **Partially committed** – after the final statement has been executed.
- **Failed** -- after the discovery that normal execution can no longer proceed.
- **Aborted** – after the transaction has been rolled back and the database restored to its state prior to the start of the transaction. Two options after it has been aborted:
  - Restart the transaction
    - Can be done only if no internal logical error
  - Kill the transaction
- **Committed** – after successful completion.

# Transaction State

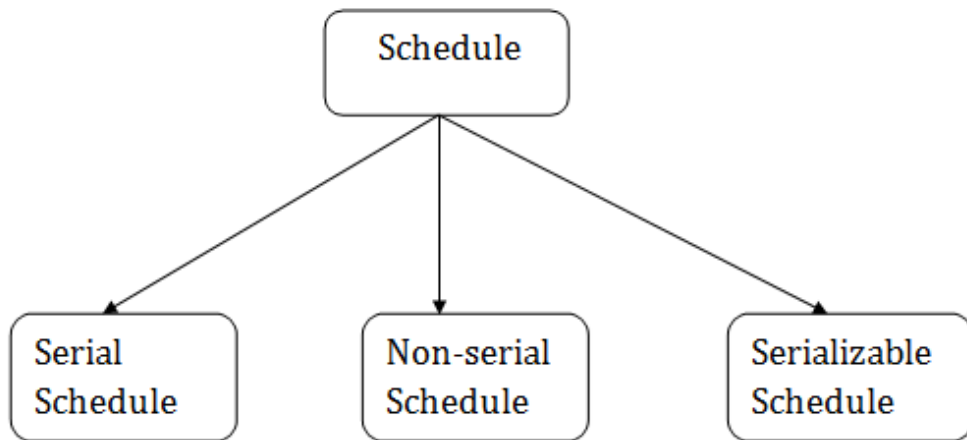


# Concurrent Executions

- Multiple transactions are allowed to run concurrently in the system. Advantages are:
  - **Increased processor and disk utilization**, leading to better transaction *throughput*
  - **Reduced average response time** for transactions: short transactions need not wait behind long ones.
- **Concurrency control schemes** – mechanisms to achieve isolation
  - That is, to control the interaction among the concurrent transactions in order to prevent them from destroying the consistency of the database

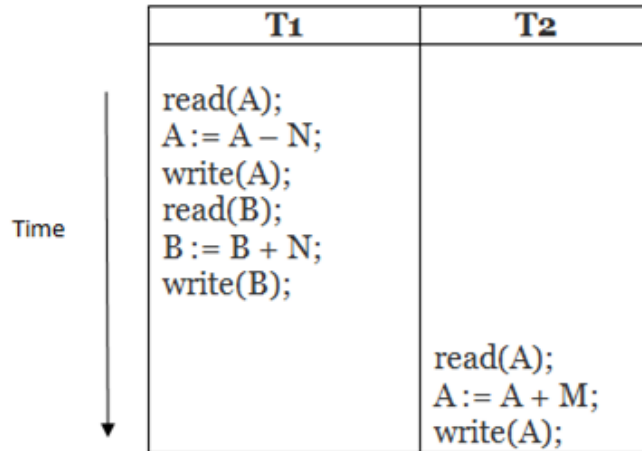
# Schedules

- It is chronological execution sequence of multiple transaction.



# Serial Schedule

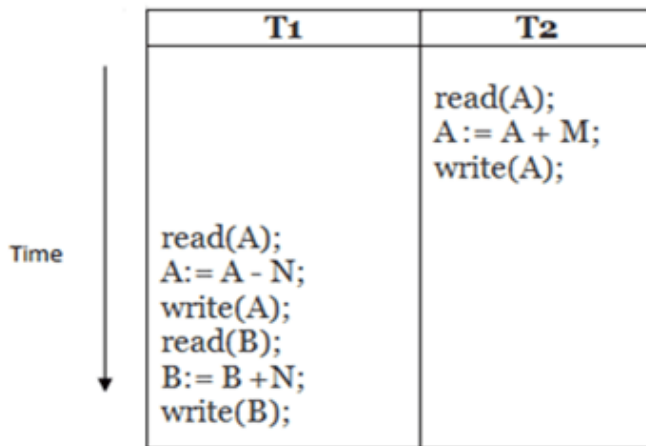
- The serial schedule is a type of schedule where one transaction is executed completely before starting another transaction. A serial schedule is indeed always consistent, but at the cost of increased waiting time for transactions.



**Schedule A**

# Serial Schedule

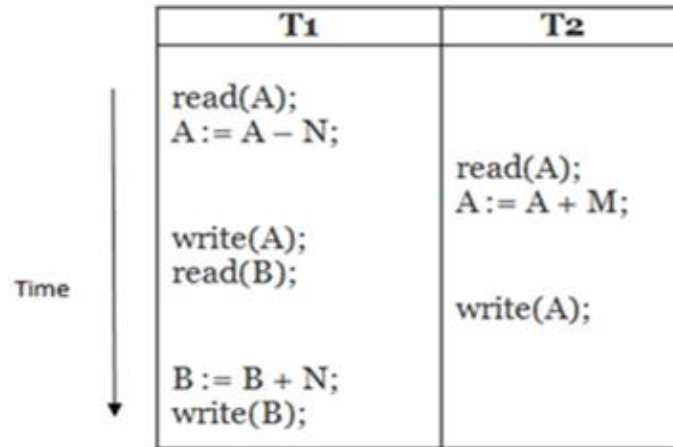
- In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.



**Schedule B**

# Parallel/Non-Serial Schedule

- If interleaving of operations is allowed, then there will be non-serial schedule.
- Parallel schedule allows multiple transactions to execute simultaneously, increasing throughput and performance.



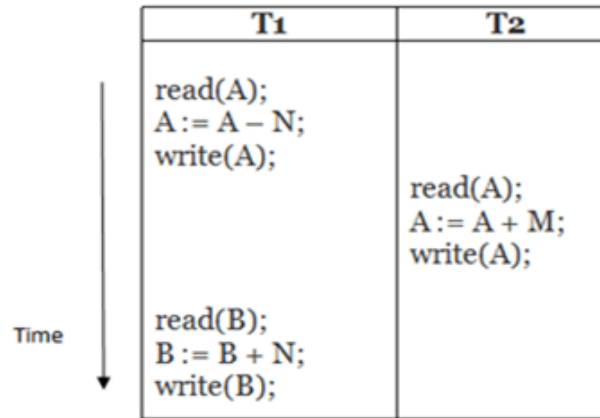
Schedule C



## Parallel/Non-Serial Schedule

- It contains many possible orders in which the system can execute the individual operations of the transactions

(d)



Schedule D

## Schedule(1)

- Let  $T_1$  transfer \$50 from  $A$  to  $B$ , and  $T_2$  transfer 10% of the balance from  $A$  to  $B$ .
- A **serial** schedule in which  $T_1$  is followed by  $T_2$  :

| $T_1$                                                                                                      | $T_2$                                                                                                                               |
|------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| read ( $A$ )<br>$A := A - 50$<br>write ( $A$ )<br>read ( $B$ )<br>$B := B + 50$<br>write ( $B$ )<br>commit | read ( $A$ )<br>$temp := A * 0.1$<br>$A := A - temp$<br>write ( $A$ )<br>read ( $B$ )<br>$B := B + temp$<br>write ( $B$ )<br>commit |

# Schedule(2)

A serial schedule where  $T_2$  is followed by  $T_1$

| $T_1$                                                                                      | $T_2$                                                                                                               |
|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| read (A)<br>$A := A - 50$<br>write (A)<br>read (B)<br>$B := B + 50$<br>write (B)<br>commit | read (A)<br>$temp := A * 0.1$<br>$A := A - temp$<br>write (A)<br>read (B)<br>$B := B + temp$<br>write (B)<br>commit |

## Schedule(3)

- Let  $T_1$  and  $T_2$  be the transactions defined previously. The following schedule is not a serial schedule, but it is *equivalent* to Schedule 1
- In Schedules 1, 2 and 3, the sum  $A + B$  is preserved.

| $T_1$                                            | $T_2$                                                         |
|--------------------------------------------------|---------------------------------------------------------------|
| read (A)<br>$A := A - 50$<br>write (A)           | read (A)<br>$temp := A * 0.1$<br>$A := A - temp$<br>write (A) |
| read (B)<br>$B := B + 50$<br>write (B)<br>commit | read (B)<br>$B := B + temp$<br>write (B)<br>commit            |

# Schedule

- The following concurrent schedule does not preserve the value of  $(A + B)$ .

| $T_1$                                                                     | $T_2$                                                                                 |
|---------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| read ( $A$ )<br>$A := A - 50$                                             |                                                                                       |
|                                                                           | read ( $A$ )<br>$temp := A * 0.1$<br>$A := A - temp$<br>write ( $A$ )<br>read ( $B$ ) |
| write ( $A$ )<br>read ( $B$ )<br>$B := B + 50$<br>write ( $B$ )<br>commit |                                                                                       |
|                                                                           | $B := B + temp$<br>write ( $B$ )<br>commit                                            |

# Serializable schedule

- **Basic Assumption** – Each transaction preserves database consistency.
- Thus, serial execution of a set of transactions preserves database consistency.
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule. Different forms of schedule equivalence give rise to the notions of:
  1. **Conflict serializability**
  2. **View serializability**

# Conflicting Instructions

- Two operations are said to be conflicting if all conditions are satisfied:
  - They belong to different transactions
  - They operate on the same data item
  - At Least one of them is a write operation

|    |                         |                         |             |
|----|-------------------------|-------------------------|-------------|
| 1. | $I_i = \text{read}(Q)$  | $I_j = \text{read}(Q)$  | No Conflict |
| 2. | $I_i = \text{read}(Q)$  | $I_j = \text{write}(Q)$ | Conflict    |
| 3. | $I_i = \text{write}(Q)$ | $I_j = \text{read}(Q)$  | Conflict    |
| 4. | $I_i = \text{write}(Q)$ | $I_j = \text{write}(Q)$ | Conflict    |

## Conflict Serializable

- A schedule is called conflict serializable if it can be transformed into a serial schedule by swapping non-conflicting operations.



# Conflict Serializable

- Schedule 3 can be transformed into Schedule 6, a serial schedule where  $T_2$  follows  $T_1$ , by series of swaps of non-conflicting instructions. Therefore Schedule 3 is conflict serializable.

| $T_1$                 | $T_2$                 |
|-----------------------|-----------------------|
| read (A)<br>write (A) |                       |
|                       | read (A)<br>write (A) |
| read (B)<br>write (B) |                       |
|                       | read (B)<br>write (B) |

Schedule 3

| $T_1$                                          | $T_2$                                          |
|------------------------------------------------|------------------------------------------------|
| read (A)<br>write (A)<br>read (B)<br>write (B) |                                                |
|                                                | read (A)<br>write (A)<br>read (B)<br>write (B) |

Schedule 6

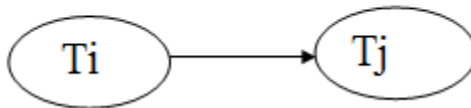
# Testing for Serializability

- Serialization Graph is used to test the Serializability of a schedule.
- Assume a schedule  $S$ . For  $S$ , we construct a graph known as precedence graph. This graph has a pair  $G = (V, E)$ , where  $V$  consists a set of vertices, and  $E$  consists a set of edges. The set of vertices is used to contain all the transactions participating in the schedule.
- The set of edges is used to contain all edges  $T_i \rightarrow T_j$  for which one of the three conditions holds:
  - Create a node  $T_i \rightarrow T_j$  if  $T_i$  executes write ( $Q$ ) before  $T_j$  executes read ( $Q$ ).
  - Create a node  $T_i \rightarrow T_j$  if  $T_i$  executes read ( $Q$ ) before  $T_j$  executes write ( $Q$ ).
  - Create a node  $T_i \rightarrow T_j$  if  $T_i$  executes write ( $Q$ ) before  $T_j$  executes write ( $Q$ ).

# Testing for Serializability

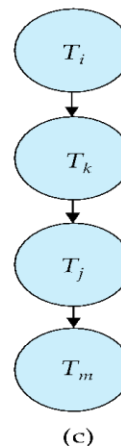
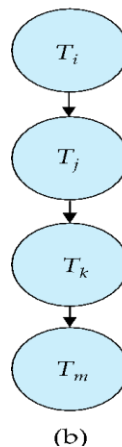
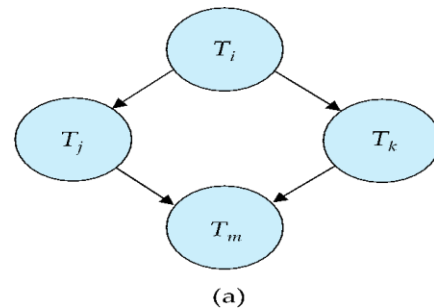
- If a precedence graph contains a single edge  $T_i \rightarrow T_j$ , then all the instructions of  $T_i$  are executed before the first instruction of  $T_j$  is executed.
- If a precedence graph for schedule  $S$  contains a cycle, then  $S$  is non-serializable. If the precedence graph has no cycle, then  $S$  is known as serializable.
- We draw an arc from  $T_i$  to  $T_j$  if the two-transaction conflict, and  $T_i$  accessed the data item on which the conflict arose earlier

**Precedence graph for Schedule S**

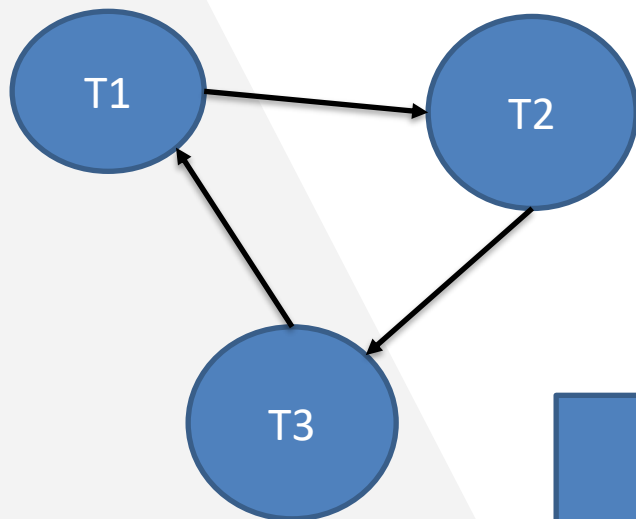


# Testing for Serializability

- If precedence graph is acyclic, the serializability order can be obtained by a *topological sorting* of the graph.
  - This is a linear order consistent with the partial order of the graph.



# Testing for Serializability



Cycle?  
Not  
serializable

| T1                        | T2                        | T3                        |
|---------------------------|---------------------------|---------------------------|
| Read(A)                   | Read(B)                   |                           |
| $A := f_1(A)$             |                           | Read(C)                   |
|                           | $B := f_2(B)$<br>Write(B) | $C := f_3(C)$<br>Write(C) |
| Write(A)                  |                           | Read(B)                   |
|                           | Read(A)<br>$A := f_4(A)$  |                           |
| Read(C)                   | Write(A)                  |                           |
| $C := f_5(C)$<br>Write(C) |                           | $B := f_6(B)$<br>Write(B) |

Schedule S1

# Serializable

Check this schedule is conflict serializable or not. If yes, then find the serial?

| T1   | T2   | T3   |
|------|------|------|
| R(X) |      |      |
|      |      | R(Y) |
|      |      | R(X) |
|      | R(Y) |      |
|      | R(Z) |      |
|      |      | W(Y) |
|      | W(Z) |      |
| R(Z) |      |      |
| W(X) |      |      |
| W(Z) |      |      |



# THE END!

Series